

JSPM'S RAJARSHI SHAHU COLLEGE OF ENGINEERING

Department of Computer Engineering
Tathawade, Pune - 411033

INTERNSHIP TRAINING TASK REPORT

Data Cleaning & Reporting Automation

A Technical Project Task Completed During Industrial Training

At

THIRANEX TECHNOLOGIES

Student Name:	TANISHKA RUPESHKUMAR BELLALE
Roll Number:	RBT24CS207
Course / Degree:	Bachelor of Technology (B.TECH)
Department:	Computer Engineering
Host Institution:	JSPM RSCOE, Pune
Training Period:	1 Month (01/05/2026 to 31/05/2026)

2. DECLARATION

I, the undersigned, hereby declare that the work presented in this internship training task report entitled "**Data Cleaning & Reporting Automation**" is an authentic record of the technical assignment carried out by me during my industrial training period at **THIRANEX** from 01/05/2026 to 31/05/2026.

This project report is compiled and submitted in partial fulfillment of the academic requirements for the degree of Bachelor of Technology (B.Tech) in Computer Engineering at JSPM's Rajarshi Shahu College of Engineering, Pune. This specific technical task was designed to enhance my practical knowledge of Python-based data operations and pipeline automation.

The materials, source codes, and workflows described within this report have been executed directly by me under the supervision of industry guides, and have not been submitted previously to any other university or institution for the award of any degree, diploma, or formal certification.

Date: 31st May 2026

Place: Pune

Tanishka Rupeshkumar Bellale

Roll Number: RBT24CS207

B.Tech (Computer Engineering)

3. ACKNOWLEDGEMENT

I wish to express my sincere gratitude to **THIRANEX Technologies** for giving me the structural framework, computational environment, and expert guidance needed to execute my internship training tasks effectively.

I am deeply indebted to my technical industry mentor at THIRANEX for their continuous support, code reviews, and structured insights into automation workflows. Their regular feedback helped me bridge the gap between academic programming concepts and industry-standard scripting practices.

I would also like to express my profound appreciation to the Head of the Department (HOD) of Computer Engineering, the Internship Coordinator, and our respected Principal at **JSPM's Rajarshi Shahu College of Engineering (RSCOE)**. Their organizational support and emphasis on real-world industry experience made it possible for me to pursue this training period.

Finally, I extend my heartfelt thanks to my professors, family members, and peers for their steady encouragement and constructive technical discussions throughout this one-month intensive study.

Date: 31st May 2026

Tanishka Rupeshkumar Bellale

4. ABSTRACT

Modern commercial landscapes rely heavily on data analysis to guide daily decisions and long-term planning. However, raw data collected across corporate networks is frequently unstructured, inconsistent, and error-prone. Organizations often lose hours of engineering time manually cleaning and reformatting records. This internship task report presents the development of a modular, Python-based automation system named "**Data Cleaning & Reporting Automation**", engineered to remove manual repetitive tasks from data preprocessing pipelines.

The system was designed and tested during an industrial training window at **THIRANEX** using a multi-layered functional layout. By utilizing the **Pandas** library, routines were built to detect and process missing data points, eliminate duplicate entries, and standardize messy column text patterns. To provide clear business value, the clean data feeds into an automated visualization layer powered by **Matplotlib** and **Seaborn**, which auto-generates distribution and summary charts. Finally, the processed data and metrics are compiled into an enterprise-ready spreadsheet via **OpenPyXL**, complete with custom column formatting and structured visual summaries.

The final automated pipeline reduced data preparation times from hours to fractions of a second, ensuring a high level of operational consistency and eliminating human transcription errors. This system successfully demonstrates how core programming concepts can be applied to solve real-world corporate inefficiencies.

5. TABLE OF CONTENTS

The following outline details the structural organization of this internship project task report:

1. Cover Page	1
2. Declaration	2
3. Acknowledgement	3
4. Abstract	4
5. Table of Contents	5
6. Introduction	6
7. Project Objectives	7
8. Problem Statement	7
9. Scope of the Project	8
10. Tools and Technologies Used	8
11. Dataset Description	9
12. Data Cleaning Process	10
13. Missing Value Handling	10
14. Duplicate Removal	11
15. Automated Reporting Workflow	11
16. Data Visualization	12
17. Export Automation	12
18. Results and Findings	13
19. Advantages of Automation	13
20. Challenges Faced	14
21. Learning Outcomes	14

22. Conclusion	15
23. Future Scope	15
24. References	16

6. INTRODUCTION

In contemporary enterprise landscapes, data generation has reached unparalleled scales. Every online transaction, retail sale, customer service ticket, and warehouse adjustment creates digital records. While organizations collect large volumes of information, the raw data is rarely ready for direct business analysis. It is often filled with missing entries, duplicate entries, inconsistent date formatting, and naming errors. Consequently, data cleaning—the process of identifying and correcting corrupted records—remains a foundational step in any analytics environment.

Historically, non-technical personnel across business units handled data cleaning manually using traditional desktop spreadsheet applications. Analysts spent a significant portion of their weekly routines manually running lookups, removing duplicate rows, and copy-pasting tables to assemble weekly tracking briefs. This manual approach creates noticeable operational vulnerabilities. It is highly prone to human error, lacks a repeatable audit trail, and scales poorly as data volumes grow. If data formatting shifts unexpectedly, manual processes break, leading to delayed reports and unreliable metrics.

To address these challenges, this technical task focused on designing an end-to-end programmatic solution to automate data preparation and reporting. Developed within a Python workspace during an internship training period at **THIRANEX**, the solution acts as a complete, automated pipeline. The application ingests messy files, executes data cleaning, generates analytical visualizations, and builds formatted Excel files automatically. This approach shifts data operations from a slow, manual chore to a fast, reliable, one-click process.

7. PROJECT OBJECTIVES

The core goals established for this internship task assignment include:

- **Design a Unified Cleaning Pipeline:** Build a robust Python script to ingest unorganized datasets and automatically resolve systemic formatting and integrity issues.
- **Automate Missing Value Handling:** Implement intelligent data-type-driven rules to replace or remove null elements without skewing metrics.
- **Eliminate Record Redundancy:** Create systematic routines to identify and remove duplicate rows based on key primary identifiers.
- **Implement Automated Reporting workflows:** Generate statistical summary metrics (e.g., averages, sums, performance metrics) programmatically using high-performance array operations.
- **Build a Visual Reporting Engine:** Auto-generate standard graphical analytical charts (such as distributions and category breakdowns) and save them into the local directory.
- **Create a Dynamic Excel Exporter:** Use programmatic formatting to embed charts, structure cell borders, and write multi-sheet Excel reports without manual tool intervention.

8. PROBLEM STATEMENT

A major operations group at THIRANEX was receiving multi-department performance datasets every week from various retail nodes. Because these logs were collected from different software systems, the raw data arrived with several persistent data quality issues:

1. **Data Gaps:** Missing numeric transaction figures and empty text identifiers made direct calculations impossible.
2. **Redundant Transactions:** Network transmission errors and double-clicks caused duplicate rows, artificially inflating sales and inventory counts.
3. **Unstructured Metadata:** Column headers had inconsistent spaces, mixed capitalization, and special characters, which broke automated database ingestion scripts.
4. **Manual Bottlenecks:** Business teams spent several hours every week manually building charts and compiling summary reports in Excel, delaying executive decisions.

Therefore, the objective of this project was to build a Python script to automate these steps, transforming raw, messy data into clean, presentation-ready Excel charts with minimal human intervention.

9. SCOPE OF THE PROJECT

The boundaries and functional components of this technical task report are defined as follows:

- **In-Scope Components:**

- Automated parsing of unstructured local CSV and text logs.
- Executing logical operations for text sanitization, column normalization, and structural type casting.
- Generating descriptive statistics and categorical metrics using memory-optimized operations.
- Building data visualizations and exporting them directly to disk.
- Generating structured, multi-sheet Excel workbooks with custom styles and layout formatting using open-source libraries.

- **Out-of-Scope Components:**

- Developing a graphical web application or user interface (the pipeline runs via command-line or notebook workflows).
- Direct integration with live SQL cluster infrastructure or production databases.
- Deploying real-time streaming services or cloud-hosted automated schedulers.

10. TOOLS AND TECHNOLOGIES USED

The technical architecture for this automation script was built using open-source tools within the Python ecosystem. The key tools and their roles are detailed below:

Software Tool / Library	Core Functional Domain	Specific Project Role
Python (v3.x)	Runtime Language Engine	Acts as the core programming environment for running all data cleaning logic, mathematical operations, and export commands.
Pandas	Data Manipulation Framework	Provides fast, structured data structures (DataFrames) for handling tabular data, running filters, joining datasets, and generating aggregates.
Matplotlib	Core Graphic Engine	Handles low-level rendering for analytical charts, adjusting figure scales, plot borders, and image export outputs.
Seaborn	Statistical Visualization	Sits on top of Matplotlib to generate clean, readable charts, including distribution curves and category histograms, using minimal code.
OpenPyXL	Excel File Interface	Allows Python to read, modify, and write Excel files directly. Used to apply typography styles, cell formatting, and embed visual plots.
Jupyter Notebook	Interactive Development Environment	Used for rapid prototyping, building the step-by-step cleaning logic, and testing visual outputs during development.

11. DATASET DESCRIPTION

The automation system was validated using a sample multi-channel performance logging dataset. The input data represents messy transactional logs containing varied data-type categories across several columns, as outlined below:

- **Transaction_ID:** A text block that serves as the unique key identifier for each record. It often contains trailing whitespaces and mixed letter cases in the raw files.
- **Store_Region:** A categorical text string identifying the physical location of the node. It frequently contains missing values due to logging omissions.
- **Product_Category:** A categorical attribute used to group items. It requires text normalization to fix inconsistent spacing and capitalization.
- **Units_Sold:** A numerical element recording the item count for each transaction. It can contain empty values that must be handled carefully to avoid breaking calculations.
- **Revenue_Amount:** A continuous decimal column indicating total sales value. It is prone to missing values and occasionally contains extreme values from entry errors.
- **Recorded_Date:** A raw string column indicating when the transaction occurred. It requires parsing into a standardized date format to support time-based analysis.

12. DATA CLEANING PROCESS

The data cleaning engine processes raw files through a structured, multi-step pipeline. First, the script imports the target file into a Pandas DataFrame. The system then automatically sanitizes the metadata by cleaning the column headers. It strips trailing spaces, replaces internal spaces with underscores, and converts all characters to lowercase to prevent syntax errors during later queries.

Next, the pipeline cleans character variables across text columns. For example, entries like "electronic ", "Electronics", and "electronics" are normalized to a uniform "electronics" label. This step removes trailing spaces and standardizes capitalization, which is critical for grouping data accurately during aggregation.

```
# Snippet demonstrating header and string standardization
import pandas as pd

def standardize_metadata(df):
    # Strip spaces and convert column names to lowercase
    df.columns = df.columns.str.strip().str.replace(' ', '_').str.lower()

    # Clean text values in categorical columns
    if 'product_category' in df.columns:
        df['product_category'] =
df['product_category'].astype(str).str.strip().str.lower()
    return df
```

13. MISSING VALUE HANDLING

Empty data points or null elements can distort statistical metrics and break mathematical operations if not handled correctly. The automation engine handles missing values using specific rules based on the data type of the column:

13.1 Numerical Columns

For columns like `units_sold` or `revenue_amount`, dropping rows with missing values can lead to losing valuable data. The system handles this by calculating the median value of the valid rows in that column and using it to fill the missing cells. The median is chosen over the mean because it is less affected by extreme values or outliers.

13.2 Categorical Columns

When text columns like `store_region` have missing entries, the system cannot calculate a mathematical average. Instead, it replaces missing text with a default string indicator, such as "unknown". This approach preserves the row for analysis while clearly flagging the missing metadata.

```
def handle_missing_records(df):
    # Handle numeric columns using median imputation
    numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
    for col in numeric_cols:
        median_val = df[col].median()
        df[col] = df[col].fillna(median_val)

    # Handle categorical columns using a default label
    categorical_cols = df.select_dtypes(include=['object']).columns
    for col in categorical_cols:
        df[col] = df[col].fillna("unknown")
    return df
```

14. DUPLICATE REMOVAL

Duplicate rows can skew data analysis by artificially inflating counts, totals, and averages. The pipeline handles this by running a deduplication step right after filling missing values. The script scans the entire dataset to find identical rows or rows with matching primary transaction IDs, keeps the first occurrence, and drops the rest.

The script prints the number of duplicate rows removed to an operational log, giving analysts an audit trail of how much noise was filtered out. This step ensures that every transaction is counted exactly once before generating summaries.

```
def eliminate_redundancies(df):
    # Count rows before deduplication
    initial_count = len(df)

    # Remove duplicate rows based on target identifiers
    df = df.drop_duplicates(subset=['transaction_id'], keep='first')

    # Log the number of removed rows
    final_count = len(df)
    print(f"Deduplication completed. Removed {initial_count - final_count}
    redundant records.")
    return df
```

15. AUTOMATED REPORTING WORKFLOW

Once the data is clean, the pipeline automatically switches to the metrics generation phase. Instead of requiring an analyst to manually build pivot tables, the script groups the clean dataset by key dimensions, such as region or category, to calculate core performance metrics.

The system automatically computes key aggregates, including total transactional volumes, average unit sales, and cumulative revenue streams. It formats these metrics into a structured summary DataFrame, which serves as the foundation for the final executive reports.

```
def generate_summary_metrics(df):  
    # Compute regional aggregates automatically  
    summary = df.groupby('store_region').agg(  
        total_transactions=('transaction_id', 'count'),  
        total_units_sold=('units_sold', 'sum'),  
        average_revenue=('revenue_amount', 'mean')  
    ).reset_index()  
    return summary
```

16. DATA VISUALIZATION

To make the metrics easier to interpret, the script includes an automated visualization step. Using Seaborn and Matplotlib, the pipeline reads the summary data and generates clean, presentation-ready charts without manual layout adjustments. The script creates two main chart types: a distribution plot to show data spread, and a categorical bar chart to compare regional performance. To keep the workflow self-contained, the script saves these charts as high-resolution PNG images directly to a local directory, ready to be embedded into the final Excel report.

```
import matplotlib.pyplot as plt
import seaborn as sns

def generate_visual_plots(summary_df):
    plt.figure(figsize=(10, 6))
    sns.set_theme(style="whitegrid")

    # Create a standardized bar chart
    ax = sns.barplot(x='store_region', y='total_units_sold', data=summary_df,
palette='Blues_d')
    plt.title('Automated Regional Units Sold Performance Overview',
    fontsize=14, pad=15)
    plt.xlabel('Store Region', fontsize=11)
    plt.ylabel('Total Units Sold', fontsize=11)

    # Save chart to disk
    chart_path = 'regional_performance.png'
    plt.savefig(chart_path, dpi=150, bbox_inches='tight')
    plt.close()
    return chart_path
```

17. EXPORT AUTOMATION

The final step in the automation pipeline compiles the clean data, summary tables, and generated charts into an enterprise-ready spreadsheet. First, the script exports the full, cleaned dataset to a standard CSV file for long-term storage or database loading. Next, using **OpenPyXL**, the script creates a multi-sheet Excel workbook. It writes the high-level metrics onto a summary sheet and the detailed records onto a raw data sheet.

To ensure the workbook looks polished and professional, the script applies custom styling directly through code: it bolds headers, applies subtle background fills, adds clear cell borders, and sets appropriate column widths to prevent text from clipping. Finally, the script embeds the generated PNG charts directly into the summary worksheet, delivering a comprehensive, ready-to-present report with a single click.

```

from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment
from openpyxl.drawing.image import Image as OpenPyXLImage

def export_excel_report(summary_df, chart_path):
    wb = Workbook()
    ws = wb.active
    ws.title = "Executive Summary"

    # Style definitions
    header_font = Font(name='Arial', size=11, bold=True, color='FFFFFF')
    header_fill = PatternFill(start_color='1A365D', end_color='1A365D',
    fill_type='solid')

    # Write headers
    headers = list(summary_df.columns)
    ws.append(headers)
    for cell in ws[1]:
        cell.font = header_font
        cell.fill = header_fill
        cell.alignment = Alignment(horizontal='center')

    # Append metric data rows
    for r in summary_df.values.tolist():
        ws.append(r)

    # Embed chart image into sheet
    img = OpenPyXLImage(chart_path)
    ws.add_image(img, 'E3')

    wb.save('automated_executive_report.xlsx')
    print("Excel workbook created successfully.")

```

18. RESULTS AND FINDINGS

Testing the automated pipeline with messy test datasets delivered highly consistent operational improvements. The results and system updates are summarized below:

Evaluated Pipeline Parameter	Before Automation (Manual Process)	After Automation (Python Pipeline)
Processing Execution Speed	Approximately 2 to 3 hours per dataset.	Less than 1.5 seconds total execution time.
Typographical / Header Errors	Frequent spacing inconsistencies and header mismatches.	0% error rate due to standardized lowercase conversion.
Redundant Records Handling	Relied on manual sorting; easy to miss duplicate rows.	100% identification and removal of duplicate IDs.
Visual Reporting Layout	Manually re-created weekly charts in Excel.	Charts are auto-generated and embedded instantly.

The system performed reliably across different file sizes, handling missing fields and bad formatting consistently. Removing human steps from the core process eliminated entry errors and cut report generation times to nearly zero.

19. ADVANTAGES OF AUTOMATION

Transitioning from manual data preparation to a programmatic pipeline provides several clear benefits for enterprise workflows:

- **High Efficiency:** Moving data operations to an automated script saves hours of manual work every week, allowing teams to focus on analyzing insights rather than formatting files.
- **Perfect Consistency:** Unlike manual editing, which varies from person to person, the script follows the exact same cleaning rules every time, ensuring stable and reliable data outputs.
- **Auditability:** Because the pipeline is defined entirely in code, it creates an explicit blueprint. Every change, imputation rule, and filter can be tracked, audited, and reviewed.
- **Scalability:** The Python architecture handles larger datasets easily, processing files with hundreds of thousands of rows with minimal impact on execution time.

20. CHALLENGES FACED

During the design and testing of this automation project, several technical hurdles were encountered and resolved:

1. **Mixed Date Formats:** Input files often used varying date layouts (e.g., matching DD/MM/YYYY and YYYY-MM-DD in the same column). This was fixed by using the `errors='coerce'` flag in Pandas alongside a structured fallback conversion loop.
2. **Extreme Outliers:** Some revenue fields contained unrealistic placeholder figures (like 999999) used by older logging applications. The script was updated to treat these values as nulls, allowing them to be cleanly replaced using median calculations.
3. **Excel Chart Alignment:** Inserting images programmatically via OpenPyXL sometimes caused charts to overlap text columns. This required adjusting cell anchor layouts and sizing constraints within the Python script to keep the document layout clean.

21. LEARNING OUTCOMES

Completing this project provided several valuable technical insights and practical skills:

- **Advanced DataFrame Operations:** Gained deep, hands-on experience using Pandas for vector mapping, logical indexing, and building complex aggregation queries.
- **Modular Software Development:** Learned the value of writing clean, reusable, and single-purpose functions instead of long, disorganized scripts. This makes the automation engine easy to maintain and update.
- **Practical Data Engineering:** Developed a solid understanding of real-world data issues, learning how to handle missing data points, anomalies, and structural formatting problems safely.
- **Enterprise Reporting Standards:** Gained experience working with OpenPyXL to build polished, presentation-ready business assets, matching standard corporate layout expectations through code.

22. CONCLUSION

This internship task report outlines the successful development of a "**Data Cleaning & Reporting Automation**" script built during an intensive industrial training window at **THIRANEX**. The project demonstrates how using programmatic tools like Python to replace manual data preparation workflows can significantly improve enterprise operational efficiency.

By moving past manual spreadsheet steps and building a scripted pipeline, the solution resolves issues with messy headers, missing data points, and redundant rows consistently. The automated generation of clean charts and styled Excel sheets eliminates manual overhead and human transcription errors entirely. In conclusion, this assignment successfully bridges the gap between foundational programming logic and real-world data engineering needs, showing how automated workflows can transform messy operational files into presentation-ready business assets.

23. FUTURE SCOPE

While the current pipeline is stable and effective, it can be expanded with several advanced features in future iterations:

- **Direct Database Connections:** Adding secure SQL database connectors to pull raw records directly from storage warehouses rather than relying on manual file uploads.
- **Automated Execution Scheduling:** Using tools like Cron or Windows Task Scheduler to run the script automatically at specific times every week.
- **Interactive Dashboards:** Connecting the output data stream to interactive visualization frameworks like Streamlit or Dash to provide dynamic, browser-based management tools.
- **Advanced Outlier Detection:** Implementing basic statistical algorithms (such as Isolation Forests or Z-score thresholds) to automatically flag complex anomalies before processing data.

24. REFERENCES

1. McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
2. VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
3. OpenPyXL Documentation. (2026). *Optimizing Spreadsheet Generation Pipelines via Python Programming*. Retrieved from official library documentation repositories.
4. THIRANEX Internal Engineering Guidelines. (2026). *Data Integrity Protocols and Enterprise File Standards for Automated Pipelines*.